

LAPORAN PRAKTIKUM STRUKTUR DATA

Sorting 2

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Sorting ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Sorting dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa Jenis dan implementasi Sorting , serta operasi – operasi dasar yang terdapat pada Sorting.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 28 Mei 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	2
PENDAHULUAN	2
1.1. Latar Belakang	2
1.2. Tujuan Praktikum	2
1.3. Manfaat Praktikum.....	2
BAB II	3
PEMBAHASAN.....	3
2.1. Deskripsi Materi Sorting.....	3
2.2. ShellSort_2511532005.....	4
2.3. QuickSort_2511532005.....	5
2.4. MergeSort_2511532005	6
2.5. BubbleSortGUI_2511532005.....	8
2.6. Analisis Praktikum.....	10
BAB III.....	12
KESIMPULAN DAN SARAN.....	12
3.1 Kesimpulan	12
3.2. Saran.....	12
DAFTAR PUSTAKA	13

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam pengolahan data, proses *sorting* merupakan salah satu operasi yang sangat penting karena dapat mempermudah pencarian, pengelompokkan, dan analisis data. Berbagai algoritma *sorting* telah dikembangkan dengan karakteristik dan Tingkat efisiensi yang berbeda-beda.

Pada praktikum Struktur Data pekan 8 ini dipelajari beberapa algoritma *sorting*, yaitu *shell sort*, *quick sort*, dan *merge sort*. Selain itu, dilakukan juga implementasi *bubble sort* dalam bentuk *Graphical User Interface* (GUI) sehingga proses pengurutan dapat diamati secara visual langkah demi langkah. Dengan memahami berbagai algoritma *sorting* tersebut, mahasiswa dapat memilih metode yang paling sesuai berdasarkan kebutuhan dan ukuran data yang diolah.

1.2. Tujuan Praktikum

1. Memahami konsep dasar *sorting* pada Struktur Data.
2. Mengimplementasikan algoritma *Shell Sort*, *Quick Sort*, dan *Merge Sort* menggunakan Java.
3. Membuat visualisasi *Bubble Sort* menggunakan GUI Java Swing.

4. Menganalisis proses dan hasil pengurutan data menggunakan berbagai algoritma sorting.

1.3.Manfaat Praktikum

1. Menambah pemahaman mengenai struktur data dan algoritma sorting.
2. Melatih kemampuan pemrograman menggunakan bahasa Java.
3. Mengetahui perbedaan cara kerja setiap algoritma *sorting*.
4. Memahami penggunaan Java Swing dalam pembuatan GUI.
5. Menjadi dasar dalam pengembangan aplikasi yang membutuhkan pengolahan data terurut.

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Sorting

Sorting merupakan penyusunan data ke dalam sebuah urutan tertentu. Sorting banyak sekali digunakan untuk mengolah data karena dapat mempermudah pencarian dan pengelompokan data.

Berikut beberapa jenis sorting yang dipelajari pada praktikum ini:

a. Shell Sort

Algoritma pengurutan yang membandingkan dan menukar elemen yang berjarak jauh terlebih dahulu, lalu secara bertahap mengurangi jarak tersebut hingga menjadi 1.

b. Quick Sort

Metode sorting yang bekerja dengan memilih sebuah pivot dan bagi elemen yang lebih kecil dan lebih besar untuk diurutkan secara rekursif, pilihan pivot mempengaruhi kinerja.

c. Merge Sort

Metode sorting yang bekerja dengan membagi data awal menjadi beberapa bagian kecil yang lebih mudah diurutkan, kemudian menyatukan semua bagian tersebut Kembali pada saat pengurutan.

Jenis – jenis tersebut termasuk algoritma sorting sederhana yang mudah dipahami dan diimplementasikan.

2.2. ShellSort_2511532005

Berikut kode program Shell Sort yang di pelajari di kelas praktikum :

```

1 package Pekan8_2511532005;
2
3 public class ShellSort_2511532005 {
4     public static void ShellSort_2511532005(int[] A) {
5         int n_2005 = A.length;
6         int gap = n_2005 / 2;
7         while (gap > 0) {
8             for (int i_2005 = gap; i_2005 < n_2005; i_2005++) {
9                 int temp = A[i_2005];
10                int j_2005 = i_2005;
11                while (j_2005 >= gap && A[j_2005 - gap] > temp) {
12                    A[j_2005] = A[j_2005 - gap];
13                    j_2005 = j_2005 - gap;
14                }
15                A[j_2005] = temp;
16            }
17            gap = gap / 2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int[] data_2005 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23        System.out.print("Sebelum : ");
24        printArray(data_2005);
25        ShellSort_2511532005(data_2005);
26
27        System.out.print("Sesudah (Shell Short) : ");
28        printArray(data_2005);
29    }
30    public static void printArray(int[] arr) {
31        for (int i_2005 : arr) System.out.print(i_2005 + " ");
32        System.out.println();
33    }
34 }
35

```

Gambar 2.1. Kode Program ShellSort_2511532005.

Kode program dan fungsi nya :

- Method digunakan untuk melakukan proses pengurutan array menggunakan algoritma Shell Sort.
- Int n_2005 = A.length; digunakan untuk menentukan panjang array.
- Int gap = n_2005 / 2; untuk menentukan nilai awal gap sebesar setengah Panjang array.
- While (gap > 0); selama gap lebih besar dari 0, dilakukan proses perbandingan dan perpindahan data.
- While (j_2005 >= gap && A[j_2005 - gap] > temp); data dibandingkan dengan elemen yang berjarak gap.
- gap = gap / 2; setelah itu datu tahap selesai, gap diperkecil menjadi setengah dari sebelumnya.

- g. Method `printarray()` digunakan untuk menampilkan isi array sebelum dan sesudah pengurutan.

2.3. QuickSort_2511532005.

Berikut kode program Quick Sort yang di pelajari di kelas praktikum :

```

1 package Pekan8_2511532005;
2
3 public class QuickSort_2511532005 {
4     static void swap_2005(int[] arr, int i_2005, int j_2005) {
5         int temp = arr[i_2005];
6         arr[i_2005] = arr[j_2005];
7         arr[j_2005] = temp;
8     }
9     // metode tambahan untuk mengatur pivot menggunakan median of three
10    static void medianOfThree_2005(int[] arr, int low_2005, int high_2005) {
11        int mid_2005 = low_2005 + (high_2005 - low_2005) / 2;
12
13        // urutkan elemen low, mid, dan high
14        if (arr[low_2005] > arr[mid_2005]) {
15            swap_2005(arr, low_2005, mid_2005);
16        }
17        if (arr[low_2005] > arr[high_2005]) {
18            swap_2005(arr, low_2005, high_2005);
19        }
20        if (arr[mid_2005] > arr[high_2005]) {
21            swap_2005(arr, mid_2005, high_2005);
22        }
23        swap_2005(arr, mid_2005, high_2005);
24    }
25    static int partition_2005(int[] arr, int low_2005, int high_2005) {
26        // panggil fungsi medianOfThree sebelum menentukan pivot
27        medianOfThree_2005(arr, low_2005, high_2005);
28        int pivot_2005 = arr[high_2005]; // sekarang arr[high_2005] sudah berisi nilai media
29        int i_2005 = (low_2005 - 1);
30
31        for (int j_2005 = low_2005; j_2005 <= high_2005 - 1; j_2005++) {
32            // jika elemen saat ini lebih kecil dari atau sama dengan pivot
33            if (arr[j_2005] < pivot_2005) {
34                // increment indeks elemen yang lebih kecil
35                i_2005++;
36                swap_2005(arr, i_2005, j_2005);
37            }
38        }
39        swap_2005(arr, i_2005 + 1, high_2005);
40
41        return (i_2005 + 1);
42    }
43    static void QuickSort_2511532005(int[] arr, int low_2005, int high_2005) {
44        if (low_2005 < high_2005) {
45            int pi_2005 = partition_2005(arr, low_2005, high_2005);
46            QuickSort_2511532005(arr, low_2005, pi_2005 - 1);
47            QuickSort_2511532005(arr, pi_2005 + 1, high_2005);
48        }
49    }
50
51    public static void printArr_2005(int[] arr) {
52        for (int i_2005 = 0; i_2005 < arr.length; i_2005++) {
53            System.out.print(arr[i_2005] + " ");
54        }
55        System.out.println();
56    }
57
58    public static void main(String[] args) {
59        int[] arr = { 10, 7, 8, 9, 1, 5 };
60        int N_2005 = arr.length;
61        System.out.print(" Data sebelum diurutkan : ");
62        printArr_2005(arr);
63
64        QuickSort_2511532005(arr, 0, N_2005 - 1);
65
66        System.out.print("Data terurut quicsort :");
67        printArr_2005(arr);
68    }
69 }
70
71 }
72

```

Gambar 2.2. Kode Program QuickSort_2511532005.

Kode program dan fungsinya :

- Swap_2005(); digunakan untuk menukar posisi dua elemen array.
- medianOfThree_2005() digunakan untuk menentukan pivot menggunakan metode Median of Three, yaitu memilih nilai Tengah dari elemen pertama, Tengah, dan terakhir.
- Partition_2005(); berfungsi membagi array berdasarkan pivot.
- QuickSort_2511532005(); merupakan fungsi rekursif yang melakukan proses pengurutan pada bagian kiri dan kanan pivot.

2.4. MergeSort_2511532005

```

1 package Pekan8_2511532005;
2
3 public class MergeSort_2511532005 {
4
5     void merge_2005(int arr[], int l_2005, int m_2005, int r_2005) {
6
7         // Find sizes of two subarrays to be merged
8         int n1_2005 = m_2005 - l_2005 + 1;
9         int n2_2005 = r_2005 - m_2005;
10
11         /* Create temp arrays */
12         int L_2005[] = new int[n1_2005];
13         int R_2005[] = new int[n2_2005];
14
15         /* Copy data to temp arrays */
16         for (int i_2005 = 0; i_2005 < n1_2005; ++i_2005)
17             L_2005[i_2005] = arr[l_2005 + i_2005];
18
19         for (int j_2005 = 0; j_2005 < n2_2005; ++j_2005)
20             R_2005[j_2005] = arr[m_2005 + 1 + j_2005];
21
22         int i_2005 = 0, j_2005 = 0;
23
24         // Initial index of merged subarray array
25         int k_2005 = l_2005;
26
27         while (i_2005 < n1_2005 && j_2005 < n2_2005) {
28             if (L_2005[i_2005] <= R_2005[j_2005]) {
29                 arr[k_2005] = L_2005[i_2005];
30                 i_2005++;
31             } else {
32                 arr[k_2005] = R_2005[j_2005];
33                 j_2005++;
34             }
35             k_2005++;
36         }
37
38         /* Copy remaining elements of L[] if any */
39         while (i_2005 < n1_2005) {

```

```

40     arr[k_2005] = L_2005[i_2005];
41     i_2005++;
42     k_2005++;
43 }
44
45 /* Copy remaining elements of R[] if any */
46 while (j_2005 < n2_2005) {
47     arr[k_2005] = R_2005[j_2005];
48     j_2005++;
49     k_2005++;
50 }
51 }
52
53 void sort_2005(int arr[], int l_2005, int r_2005) {
54     if (l_2005 < r_2005) {
55         // Find the middle point
56         int m_2005 = (l_2005 + r_2005) / 2;
57
58         // Sort first and second halves
59         sort_2005(arr, l_2005, m_2005);
60         sort_2005(arr, m_2005 + 1, r_2005);
61
62         // Merge the sorted halves
63         merge_2005(arr, l_2005, m_2005, r_2005);
64     }
65 }
66
67 /* A utility function to print array of size n */
68 static void printArray_2005(int arr[]) {
69     int n_2005 = arr.length;
70     for (int i_2005 = 0; i_2005 < n_2005; ++i_2005)
71         System.out.print(arr[i_2005] + " ");
72     System.out.println();
73 }
74
75 }
76
77
78 public static void main(String args[]) {
79     int arr[] = {12, 11, 13, 5, 6, 7};
80
81     System.out.println("Sebelum terurut");
82     printArray_2005(arr);
83
84     MergeSort_2511532005 ob_2005 = new MergeSort_2511532005();
85     ob_2005.sort_2005(arr, 0, arr.length - 1);
86
87     System.out.println("\nSesudah Terurut menggunakan merge Sort");
88     printArray_2005(arr);
89 }
90
91 }

```

Gambar 2.3. Kode Program MergeSort_2511532005.

Kode program dan fungsinya :

- Method merge_2005 berfungsi menggabungkan dua sub-array yang telah terurut menjadi satu array yang terurut.
- Sort_2005(); berfungsi membagi array secara rekursif hingga menjadi bagian-bagian kecil.

2.5. BubbleSortGUI_2511532005

```

1 package Pekan8_2511532005;
2
3 import java.awt.BorderLayout;
4
5
6 public class BubbleSortGUI_2511532005 extends JFrame {
7     private static final long serialVersionUID = 1L;
8     private int[] array_2005;
9     private JLabel[] labelArray_2005;
10    private JButton stepButton_2005, resetButton_2005, setButton_2005;
11    private JTextField inputField_2005;
12    private JPanel panelArray_2005;
13    private JTextArea stepArea_2005;
14    private int i_2005 = 0, j_2005; // i_2005 diinisialisasi 0 agar sinkron dengan setArrayFromInput
15    private boolean sorting_2005 = false;
16    private int stepCount_2005 = 1;
17
18    /**
19     * Create the frame.
20     */
21    public BubbleSortGUI_2511532005() {
22        setTitle("Bubble Sort Langkah per Langkah"); // Disesuaikan dengan logika Bubble Sort di bawah
23        setSize(750, 400);
24        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25        setLocationRelativeTo(null);
26        setLayout(new BorderLayout());
27
28        // Panel input
29        JPanel inputPanel_2005 = new JPanel(new FlowLayout());
30        inputField_2005 = new JTextField(30);
31        setButton_2005 = new JButton("Set Array");
32        inputPanel_2005.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
33        inputPanel_2005.add(inputField_2005);
34        inputPanel_2005.add(setButton_2005);
35
36        // Panel array visual
37        panelArray_2005 = new JPanel();
38        panelArray_2005.setLayout(new FlowLayout());
39
40        panelArray_2005.setLayout(new FlowLayout());
41
42        // Panel kontrol
43        JPanel controlPanel_2005 = new JPanel();
44        stepButton_2005 = new JButton("Langkah Selanjutnya");
45        resetButton_2005 = new JButton("Reset");
46        stepButton_2005.setEnabled(false);
47        controlPanel_2005.add(stepButton_2005);
48        controlPanel_2005.add(resetButton_2005);
49
50        // Area teks untuk log langkah-langkah
51        stepArea_2005 = new JTextArea(8, 60);
52        stepArea_2005.setEditable(false);
53        stepArea_2005.setFont(new Font("Monospaced", Font.PLAIN, 14));
54        JScrollPane scrollPane_2005 = new JScrollPane(stepArea_2005);
55
56        // Tambahkan panel ke frame
57        add(inputPanel_2005, BorderLayout.NORTH);
58        add(panelArray_2005, BorderLayout.CENTER);
59        add(controlPanel_2005, BorderLayout.SOUTH);
60        add(scrollPane_2005, BorderLayout.EAST);
61
62        // Event Set Array
63        setButton_2005.addActionListener(e -> setArrayFromInput_2005());
64
65        // Event Langkah Selanjutnya
66        stepButton_2005.addActionListener(e -> performStep_2005());
67
68        // Event reset
69        resetButton_2005.addActionListener(e -> reset_2005());
70    }
71
72    private void setArrayFromInput_2005() {
73        String text = inputField_2005.getText().trim();
74        if (text.isEmpty()) return;
75        String[] parts = text.split(",");
76        array_2005 = new int[parts.length];
77        try {
78

```

```

80     for (int k = 0; k < parts.length; k++) {
81         array_2005[k] = Integer.parseInt(parts[k].trim());
82     }
83     } catch (NumberFormatException e) {
84         JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
85             + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
86         return;
87     }
88     i_2005 = 0;
89     j_2005 = 0;
90     stepCount_2005 = 1;
91     sorting_2005 = true;
92     stepButton_2005.setEnabled(true);
93     stepArea_2005.setText("");
94     panelArray_2005.removeAll();
95     labelArray_2005 = new JLabel[array_2005.length];
96     for (int k = 0; k < array_2005.length; k++) {
97         labelArray_2005[k] = new JLabel(String.valueOf(array_2005[k]));
98         labelArray_2005[k].setFont(new Font("Arial", Font.BOLD, 24));
99         labelArray_2005[k].setOpaque(true);
100        labelArray_2005[k].setBackground(Color.WHITE);
101        labelArray_2005[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
102        labelArray_2005[k].setPreferredSize(new Dimension(50, 50));
103        labelArray_2005[k].setHorizontalAlignment(SwingConstants.CENTER);
104        panelArray_2005.add(labelArray_2005[k]);
105    }
106    panelArray_2005.revalidate();
107    panelArray_2005.repaint();
108 }
109
110 private void performStep_2005() {
111     if (!sorting_2005 || i_2005 >= array_2005.length - 1) {
112         sorting_2005 = false;
113         stepButton_2005.setEnabled(false);
114         JOptionPane.showMessageDialog(this, "Sorting selesai!");
115         return;
116     }
117
118     resetHighlights_2005(); // Perbaikan: Memanggil method pembantu dengan benar
119
120     StringBuilder stepLog = new StringBuilder();
121     labelArray_2005[j_2005].setBackground(Color.CYAN);
122     labelArray_2005[j_2005 + 1].setBackground(Color.CYAN);
123
124     if (array_2005[j_2005] > array_2005[j_2005 + 1]) {
125         // Swap
126         int temp = array_2005[j_2005];
127         array_2005[j_2005] = array_2005[j_2005 + 1];
128         array_2005[j_2005 + 1] = temp;
129         labelArray_2005[j_2005].setBackground(Color.RED);
130         labelArray_2005[j_2005 + 1].setBackground(Color.RED);
131         stepLog.append("Langkah ").append(stepCount_2005).append(": Menukar elemen ke-")
132             .append(j_2005).append(" ").append(array_2005[j_2005 + 1]).append(" dengan ke-")
133             .append(j_2005 + 1).append(" ").append(array_2005[j_2005]).append("\n");
134     } else {
135         stepLog.append("Langkah ").append(stepCount_2005).append(": Tidak ada pertukaran antara ke-")
136             .append(j_2005).append(" ").append(array_2005[j_2005]).append(" dan ke-").append(j_2005 + 1).append("\n");
137     }
138
139     stepLog.append("Hasil: ").append(arrayToString_2005(array_2005)).append("\n\n");
140     stepArea_2005.append(stepLog.toString());
141     updateLabels_2005();
142
143     j_2005++;
144     if (j_2005 >= array_2005.length - i_2005 - 1) {
145         j_2005 = 0;
146         i_2005++;
147     }
148     stepCount_2005++;
149     if (i_2005 >= array_2005.length - 1) {
150
151         sorting_2005 = false;
152         stepButton_2005.setEnabled(false);
153         JOptionPane.showMessageDialog(this, "Sorting selesai!");
154     }
155 }
156
157 private void updateLabels_2005() {
158     for (int k = 0; k < array_2005.length; k++) {
159         labelArray_2005[k].setText(String.valueOf(array_2005[k]));
160     }
161 }
162
163 private void resetHighlights_2005() { // Perbaikan: Nama method disinkronkan
164     for (JLabel label : labelArray_2005) {
165         label.setBackground(Color.WHITE);
166     }
167 }
168
169 private void reset_2005() { // Perbaikan: Nama method disinkronkan
170     inputField_2005.setText("");
171     panelArray_2005.removeAll();
172     panelArray_2005.revalidate();
173     panelArray_2005.repaint();
174     stepArea_2005.setText("");
175     stepButton_2005.setEnabled(false);
176     sorting_2005 = false;
177     i_2005 = 0;
178     j_2005 = 0;
179     stepCount_2005 = 1;
180 }
181
182 private String arrayToString_2005(int[] arr) { // Perbaikan: Nama method disinkronkan
183     StringBuilder sb = new StringBuilder();
184     for (int k = 0; k < arr.length; k++) {
185         sb.append(arr[k]);

```

```

186         if (k < arr.length - 1) sb.append(", ");
187     }
188     return sb.toString();
189 }
190
191 public static void main(String[] args) {
192     SwingUtilities.invokeLater(() -> {
193         BubbleSortGUI_2511532005 gui = new BubbleSortGUI_2511532005();
194         gui.setVisible(true);
195     });
196 }
197 }

```

Gambar 2.4. Kode Program BubbleSortGUI_2511532005.

Kode program dan fungsi:

- a. `setArrayFromInput_2005()`; membaca input pengguna kemudian mengubahnya menjadi array integer.
- b. `performStep_2005()`; melakukan satu langkah Bubble Sort setiap kali tombol “langkah selanjutnya” ditekan.
- c. `updateLabels_2005()`; memperbarui tampilan array pada GUI setelah terjadi perubahan data.
- d. `resetHighlights_2005()`; mengembalikan warna elemen array menjadi putih.
- e. `Reset_2005()`; menghapus seluruh data dan mengembalikan kondisi program ke keadaan awal.

2.6. Analisis Praktikum

Berdasarkan hasil praktikum yang telah dilakukan, seluruh algoritma sorting yang diimplementasikan, yaitu Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort GUI, berhasil mengurutkan data dari kondisi tidak terurut menjadi terurut secara ascending. Shell Sort menunjukkan peningkatan efisiensi dibandingkan Bubble Sort karena menggunakan nilai gap yang mempercepat proses perpindahan elemen. Quick Sort bekerja sangat cepat dengan metode Divide and Conquer serta pemilihan pivot menggunakan Median of Three sehingga proses pengurutan menjadi lebih optimal. Merge Sort juga menghasilkan pengurutan yang stabil dan konsisten dengan membagi array menjadi bagian-bagian kecil kemudian menggabungkannya kembali secara

terurut, meskipun membutuhkan memori tambahan selama proses berlangsung. Selain itu, implementasi Bubble Sort dalam bentuk GUI memberikan visualisasi langkah demi langkah yang memudahkan pengguna dalam memahami proses perbandingan dan pertukaran elemen pada algoritma sorting. Dari praktikum ini dapat disimpulkan bahwa setiap algoritma memiliki karakteristik dan tingkat efisiensi yang berbeda, sehingga pemilihan algoritma perlu disesuaikan dengan kebutuhan dan ukuran data yang akan diolah.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort berhasil diimplementasikan menggunakan bahasa pemrograman Java dan mampu mengurutkan data dengan baik sesuai urutan yang diinginkan. Setiap algoritma memiliki cara kerja dan karakteristik yang berbeda, di mana Shell Sort meningkatkan efisiensi melalui penggunaan gap, Quick Sort dan Merge Sort memanfaatkan metode Divide and Conquer untuk menghasilkan performa yang lebih baik, sedangkan Bubble Sort memiliki mekanisme yang lebih sederhana dan mudah dipahami. Selain itu, implementasi Bubble Sort dalam bentuk GUI memberikan visualisasi proses pengurutan secara interaktif sehingga membantu dalam memahami konsep sorting secara lebih jelas. Melalui praktikum ini, pemahaman mengenai algoritma pengurutan data dan penerapannya dalam pemrograman menjadi semakin baik.

3.2. Saran

Untuk praktikum selanjutnya, disarankan agar dilakukan pengujian menggunakan jumlah data yang lebih besar dan variasi data yang lebih beragam sehingga perbedaan performa antar algoritma dapat diamati dengan lebih jelas. Selain itu, visualisasi GUI dapat dikembangkan dengan menambahkan animasi atau grafik yang lebih menarik agar proses sorting lebih mudah dipahami. Mahasiswa juga disarankan untuk mempelajari analisis kompleksitas waktu dan

penggunaan memori dari setiap algoritma secara lebih mendalam sehingga dapat menentukan algoritma yang paling sesuai untuk berbagai kebutuhan pengolahan data. Dengan demikian, pemahaman terhadap struktur data dan algoritma dapat semakin meningkat serta dapat diterapkan dalam pengembangan program yang lebih kompleks.

DAFTAR PUSTAKA

- Algoritma Merge Sort: Pengertian, Cara Kerja dan Implementasi.* (2023). Retrieved from KantinIT.
- Chen, S. (2025). *Algoritma Pengurutan Shell dengan CONTOH.* Retrieved from Guru99.
- Rahmaratih. (2023). *Algoritma Quick Sort: Pengertian, Kelebihan dan Contoh.* Retrieved from DosenIT.com.

TUGAS PRAKTIKUM 8

1. Perintah Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

2. ADT Lagu

```

1 package Pekan8_2511532005;
2
3 public class Lagu_2511532005 {
4     String judul_2005;
5     String penyanyi_2005;
6     int durasi_2005;
7
8     // Konstruktor
9     public Lagu_2511532005 (String judul_2005, String penyanyi_2005, int durasi_2005) {
10         this.judul_2005 = judul_2005;
11         this.penyanyi_2005 = penyanyi_2005;
12         this.durasi_2005 = durasi_2005;
13     }
14
15     // Getter untuk durasi
16     public int getDurasi_2005() {
17         return this.durasi_2005;
18     }
19
20     // Method untuk menampilkan data lagu
21     public void tampilkan_2005() {
22         System.out.println(judul_2005 + " - " + durasi_2005 + " detik");
23     }
24 }
25

```

Penjelasan kode :

- a. Deklarasi kelas : cetakan atau template membuat objek lagu.
- b. Deklarasi atribut : ada judul, penyanyi, dan durasi yang menyimpan data lagu dalam playlist.
- c. Konstruktor digunakan untuk memberikan nilai awal pada atribut saat objek dibuat
- d. Getter digunakan mengambil atau mengembalikan nilai.
- e. Lalu ada method tampilkan itu untuk menampilkan informasi lagu ke layar.

3. Drive Sorting

```

1 package Pekan8_2511532005;
2 import java.util.Scanner;
3
4 import java.util.Scanner;
5
6 public class Sorting_2511532005 {
7     // Array untuk menyimpan maksimal 20 lagu
8     Lagu_2511532005[] dataLagu_2005 = new Lagu_2511532005[20];
9     int n = 0;
10    Scanner scanner = new Scanner(System.in);
11
12    // Method untuk input data secara MANUAL
13    public void inputData_2005() {
14        System.out.print("Berapa banyak lagu yang ingin diinput? (minimal 3): ");
15        int jumlah_2005 = scanner.nextInt();
16        scanner.nextLine(); // consume newline
17
18        // Validasi minimal 3 lagu
19        if (jumlah_2005 < 3) {
20            System.out.println("Minimal harus 3 lagu!");
21            return;
22        }
23
24        // Batasi maksimal 20 lagu
25        if (jumlah_2005 > 20) {
26            System.out.println("Maksimal 20 lagu! Hanya bisa input 20.");
27            jumlah_2005 = 20;
28        }
29
30        // Input data lagu satu per satu
31        for (int i = 0; i < jumlah_2005; i++) {
32            System.out.println("\n--- Lagu ke- " + (i + 1) + " ---");
33            System.out.print("Judul : ");
34            String judul_2005 = scanner.nextLine();
35            System.out.print("Penyanyi: ");
36            String penyanyi_2005 = scanner.nextLine();
37            System.out.print("Durasi (detik): ");
38            int durasi_2005 = scanner.nextInt();
39            scanner.nextLine(); // consume newline
40
41            dataLagu_2005[i] = new Lagu_2511532005(judul_2005, penyanyi_2005, durasi_2005);
42        }
43
44        n = jumlah_2005;
45        System.out.println("\n Data berhasil diinput!");
46    }
47
48    // Method menampilkan data
49    public void tampilData_2005() {
50        for (int i = 0; i < n; i++) {
51            System.out.println((i + 1) + " . " + dataLagu_2005[i].judul_2005 +
52                " . " + dataLagu_2005[i].durasi_2005 + " detik");
53        }
54    }
55
56    // QUICK SORT - Mengurutkan berdasarkan Durasi (Ascending)
57    int partition_2005(int low_2005, int high_2005) {
58        Lagu_2511532005 pivot = dataLagu_2005[high_2005];
59        int i = (low_2005 - 1);
60
61        for (int j = low_2005; j < high_2005; j++) {
62            if (dataLagu_2005[j].getDurasi_2005() <= pivot.getDurasi_2005()) {
63                i++;
64                Lagu_2511532005 temp = dataLagu_2005[i];
65                dataLagu_2005[i] = dataLagu_2005[j];
66                dataLagu_2005[j] = temp;
67            }
68        }
69
70        Lagu_2511532005 temp = dataLagu_2005[i + 1];
71        dataLagu_2005[i + 1] = dataLagu_2005[high_2005];
72        dataLagu_2005[high_2005] = temp;
73
74        return i + 1;
75    }
76
77    // Method utama Quick Sort (wajib pakai nama quickSort_xxxx)
78    void quickSort_2005(int low_2005, int high_2005) {
79        if (low_2005 < high_2005) {
80            int pi = partition_2005(low_2005, high_2005);
81            quickSort_2005(low_2005, pi - 1);
82            quickSort_2005(pi + 1, high_2005);
83        }
84    }
85
86    // Main Method
87    public static void main(String[] args) {
88        Sorting_2511532005 playlist = new Sorting_2511532005();
89
90        System.out.println("=== Sorting Playlist NIM: 2511532005 ===");
91        System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2");
92
93        // Input Data secara MANUAL
94        playlist.inputData_2005();
95
96        // Tampilkan Sebelum Sorting
97        System.out.println("\n Data Sebelum Sorting:");
98        playlist.tampilData_2005();
99
100       // Proses Quick Sort
101       playlist.quickSort_2005(0, playlist.n - 1);
102
103       // Tampilkan Sesudah Sorting
104       System.out.println("\n Data Setelah Quick Sort (Durasi Asc):");
105       playlist.tampilData_2005();
106   }
107 }

```

Penjelasan kode :

- a. Import scanner karena digunakan untuk menerima input dari pengguna.
- b. Variable nya ada dataLagu, n (menyimpan jumlah lagu input), dan scanner untuk membaca input pengguna.
- c. Method inputData digunakan untuk memasukkan data lagu secara manual.
- d. Method tampilkanData digunakan untuk menampilkan seluruh data lagu yang tersimpan.
- e. Method partition digunakan untuk bagian utama quicksort yang bertugas menentukan pivot.
- f. Method quicksort untuk algoritma nya si quick sort.
- g. Method main, tempat utama yang menjalankan program objek playlist, input data lagu, menampilkan data awal, menampilkan hasil.

4. Output

```
=== Sorting Playlist NIM: 2511532005 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2
Berapa banyak lagu yang ingin diinput? (minimal 3): 5

--- Lagu ke-1 ---
Judul   : love
Penyanyi: keyshia cole
Durasi (detik): 255

--- Lagu ke-2 ---
Judul   : drag path
Penyanyi: twenty one pilots
Durasi (detik): 210

--- Lagu ke-3 ---
Judul   : heaven
Penyanyi: bryan adams
Durasi (detik): 243

--- Lagu ke-4 ---
Judul   : zombie
Penyanyi: the cranberries
Durasi (detik): 305

--- Lagu ke-5 ---
Judul   : always
Penyanyi: daneil caesar
Durasi (detik): 205
```

☒ Data berhasil diinput!

 Data Sebelum Sorting:

1. love - 255 detik
2. drag path - 210 detik
3. heaven - 243 detik
4. zombie - 305 detik
5. always - 205 detik

 Data Setelah Quick Sort (Durasi Asc):

1. always - 205 detik
2. drag path - 210 detik
3. heaven - 243 detik
4. love - 255 detik
5. zombie - 305 detik